

III BCA - FRONTEND DESIGN

UNIT I: Core Web Foundations

Semantic Structure and Layout Design: HTML5 semantic elements, forms, accessibility basics, layout design using Flexbox and CSS Grid, structuring clean and maintainable web pages.

Responsive Design and Modern Styling: Responsive design using media queries (mobile-first), introduction to modern styling tools (Tailwind CSS via CDN), basics of web concepts including browsers, HTTP/HTTPS, and DNS.

This unit builds the foundation of frontend design by focusing on the first two pillars in depth — how to structure a page semantically and accessibly using HTML5, and how to lay it out and style it using Flexbox, CSS Grid, media queries, and Tailwind CSS.

SEMANTIC STRUCTURE AND LAYOUT DESIGN

HTML5 Document Structure and Semantic Elements

HTML is often called the skeleton of a web page because it defines the structural bones on which everything else — style and behaviour is hung. Every web page a browser renders begins its life as a plain text file containing HTML markup. The browser reads this text, top to bottom, and constructs an internal tree-like representation of it called the DOM — the Document Object Model.

HTML5 is the fifth and current major revision of the HTML standard. Earlier versions (HTML4 and XHTML) allowed developers to build any layout they liked using generic `<div>` containers, distinguished only by class attributes like `class="header"` or `class="footer"`. To the browser's rendering and accessibility machinery, a `div` with `class="header"` was exactly as meaningful as a `div` with `class="box3"`.

HTML5 addressed this gap by introducing a family of semantic elements — tags whose very names describe the role of the content they wrap — such as `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, and `<footer>`. This single change — replacing meaningless boxes with self-describing tags — had far-reaching consequences for accessibility, search engine optimisation, and code maintainability.

The Basic HTML5 Document Skeleton

Every HTML5 document, no matter how simple or complex, begins with the same basic skeleton. Below is the minimal structure that any web page should start from:

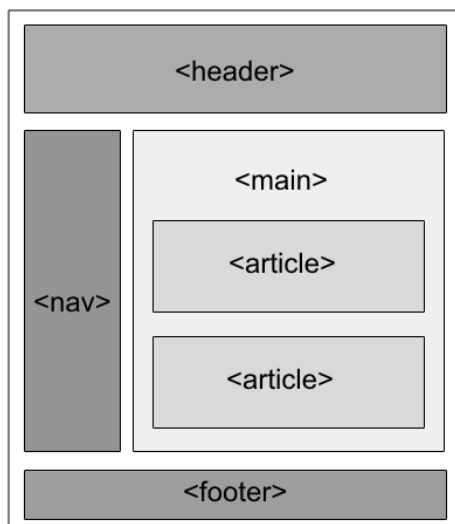
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Web Page</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>Hello, World!</h1>
    <script src="app.js" defer></script>
  </body>
</html>
```

Each line in this skeleton exists for a specific, deliberate reason:

- **DOCTYPE Declaration:** `<!DOCTYPE html>` — Forces the browser into standards mode rather than legacy quirks mode. Without it, browsers reproduce non-standard rendering bugs from the 1990s.
- **Root Element:** `<html lang="en">` — The root element. The `lang` attribute tells screen readers which pronunciation engine to use and helps search engines serve the right language version.
- **Character Encoding:** `<meta charset="UTF-8">` — Declares character encoding. UTF-8 supports virtually every written language and emoji. Without it, special characters may render as garbled symbols.
- **Viewport Meta:** `<meta name="viewport" content="width=device-width, initial-scale=1.0">` — Tells mobile browsers not to zoom out and display the page at desktop width. Essential for responsive design.
- **Page Title:** `<title>` — Sets the text that appears in the browser tab, bookmarks, search results, and screen reader announcements.
- **Script defer:** `defer` on `<script>` — Ensures the script downloads in parallel but executes only after the HTML is fully parsed, preventing the script from blocking page rendering.

Core Semantic Elements — Detailed Walkthrough

HTML5 introduced roughly a dozen new sectioning and content elements. Misusing them (for instance, using multiple `<main>` elements, or nesting an `<article>` where a plain `<div>` would have been more appropriate) can actually harm accessibility rather than help it.



`<header>`

Represents introductory content for either the page as a whole or for a section within the page. It commonly contains a site logo, the main heading, and a navigation menu. A page may legally contain more than one `<header>`, as long as each is scoped to its own sectioning element.

`<nav>`

Identifies a block of navigation links intended primarily for navigating the site or the current page. Not every group of links needs to be wrapped in `<nav>` — a handful of links inside a footer is common but does not always need marking as a distinct nav landmark. Overusing nav for every scrap of link text dilutes its usefulness as a landmark for assistive technology users.

`<main>`

Identifies the dominant, unique content of the document — the content specific to this particular page, as opposed to content repeated across every page (like the header or footer). A crucial rule: there should be only one `<main>` element visible per page at any given time. This restriction ensures assistive technology has an unambiguous target when a user chooses "skip to main content".

`<article>`

Wraps independent, self-contained content that would make sense on its own if extracted and syndicated elsewhere — a blog post, a news story, a forum post, or a product card. The test: "If I pulled this out and put it in an RSS feed or a different page, would it still make complete sense?" If yes, `<article>` is appropriate.

<section>

Represents a thematic grouping of content, generally accompanied by its own heading. An important nuance: `<section>` is not simply a semantic rename of `<div>`. If a chunk of content has no natural heading and exists purely as a styling or scripting hook, a plain `<div>` remains the correct choice.

<aside>

Contains content that is tangentially related to the surrounding content — a sidebar of related links, a pull-quote, advertising blocks, or a glossary definition box. Content in an `<aside>` is understood by assistive technology as supplementary rather than part of the primary reading flow.

<footer>

Represents the footer for its nearest ancestor sectioning content — most commonly the page itself, containing copyright notices, contact information, sitemap links, and social media links. Like `<header>`, a footer can also belong to a smaller sectioning element such as an individual article.

<figure> and <figcaption>

Together represent self-contained media — an image, diagram, code listing, or embedded video — along with an optional caption. The key semantic property: figure content could be moved to an appendix without affecting the overall meaning of the surrounding text.

<time>

Marks up a specific date or time in a way that is both human-readable in its visible text and machine-readable through its `datetime` attribute (ISO 8601 format like 2026-06-01). This allows browsers, extensions, and search engines to extract and act on dates programmatically.

Example: Semantic time element

```
<!-- Machine-readable date, human-friendly display -->  
<time datetime="2026-06-01">June 1, 2026</time>
```

<mark>, <details>, and <summary>

`<mark>` highlights text of particular relevance in the current context, most commonly to highlight search terms within a block of matched text — it renders with a yellow background by default.

The paired `<details>` and `<summary>` elements provide a native collapsible disclosure widget entirely without JavaScript:

```
<details>  
  <summary>What is your return policy?</summary>
```

```
<p>Items can be returned within 30 days of purchase,  
provided they are unused and in original packaging.</p>  
</details>
```

First Rule of ARIA: Prefer a native HTML element with built-in behaviour over reconstructing that same behaviour manually with JavaScript and ARIA attributes. `<details>/<summary>` replaces what would otherwise require custom JS, aria-expanded, and manual keyboard accessibility work.

A Full Worked Example — Semantic Blog Page

```
<body>  
  <header>  
    <h1>My Travel Blog</h1>  
    <nav>  
      <ul>  
        <li><a href="/">Home</a></li>  
        <li><a href="/about">About</a></li>  
        <li><a href="/contact">Contact</a></li>  
      </ul>  
    </nav>  
  </header>  
  
  <main>  
    <article>  
      <h2>Trekking in the Himalayas</h2>  
      <p><time datetime="2026-06-01">June 1, 2026</time></p>  
      <figure>  
          
        <figcaption>Sunrise over the Annapurna range</figcaption>  
      </figure>  
      <p>The trek began at dawn...</p>  
      <section>  
        <h3>Packing List</h3>  
        <p>Essentials for a week-long trek...</p>
```

```
</section>
</article>
<aside>
  <h3>Related Posts</h3>
  <ul><li><a href="/andes">Trekking the Andes</a></li></ul>
</aside>
</main>

<footer>
  <p>&copy; 2026 My Travel Blog. All rights reserved.</p>
</footer>
</body>
```

Two rules are worth committing to memory: A page should generally have only one `<main>` and one top-level `<h1>`. And `<section>` elements should almost always have an accompanying heading — if a block of content has no logical heading and exists purely for visual grouping, a plain `<div>` is the more honest choice.

HTML Forms

HTML Forms

A form is a collection of interactive controls that allows users to enter data and submit it — for login, registration, search, contact, payment, and more. Every form element carries both a visual presentation and a semantic role that browsers and assistive technologies rely on.

The `<form>` Element and Its Key Attributes

The `<form>` element wraps all form controls and defines where and how the submitted data travels:

```
<form action="/login" method="POST" novalidate>
  <!-- form controls go here -->
</form>
```

- **action:** action — The URL to which the form data is sent on submission. If omitted, the form submits to the current page URL.

- **method:** method — POST sends form data inside the HTTP request body (used for sensitive data, mutations). GET appends data to the URL as query parameters (used for searches).
- **novalidate:** novalidate — Disables browser-native HTML5 validation, useful when implementing custom JavaScript validation instead.
- **enctype:** enctype="multipart/form-data" — Required when the form includes a file upload input.

Input Types — The Full Vocabulary

The `<input>` element's type attribute dramatically changes both its visual appearance and its validation behaviour:

Type	Description	Example Use
text	Single-line plain text input	Name, username
email	Validates email format (user@domain.com)	Login form, newsletter
password	Masks characters as dots/asterisks	Password entry
number	Numeric keyboard on mobile, min/max/step	Age, quantity, price
tel	Numeric keyboard on mobile, no format validation	Phone number
url	Validates URL format (must start with http://)	Website field
date	Date picker widget in supporting browsers	Birthday, deadline
checkbox	Boolean toggle, can select multiple	Agree to terms
radio	One-of-group selection (same name attribute)	Gender, plan selection
file	File upload control (use multipart/form-data)	Avatar upload, PDF
range	Slider control with min/max/step	Volume, brightness

color	Colour picker widget	Theme customisation
search	Search field with clear (×) button	Site search bar
hidden	Not visible to user, submits value silently	CSRF token, ID
submit	Button that triggers form submission	Login, Register

Labels, Fieldsets, and Accessibility

Every input must have an associated label. Labels are not optional — they are how screen readers announce what an input is for, and they increase the clickable area for mouse users:

<!-- Method 1: Explicit association via for/id -->

```
<label for="email">Email Address</label>
```

```
<input type="email" id="email" name="email" required>
```

<!-- Method 2: Wrapping (implicit association) -->

```
<label>
```

```
  Email Address
```

```
  <input type="email" name="email" required>
```

```
</label>
```

<!-- Method 3: aria-label for icon-only inputs -->

```
<input type="search" aria-label="Search the site" placeholder="Search...">
```

Use <fieldset> and <legend> to group related controls — most commonly a set of radio buttons or checkboxes:

```
<fieldset>
```

```
  <legend>Preferred Contact Method</legend>
```

```
  <label><input type="radio" name="contact" value="email"> Email</label>
```

```
  <label><input type="radio" name="contact" value="phone"> Phone</label>
```

```
  <label><input type="radio" name="contact" value="post"> Post</label>
```

```
</fieldset>
```

HTML5 Built-in Validation Attributes

HTML5 provides client-side validation without JavaScript through these attributes:

- **required:** required — Field must not be empty before submission.
- **minlength / maxlength:** minlength / maxlength — Minimum and maximum character count for text inputs.
- **min / max:** min / max — Minimum and maximum values for number, date, and range inputs.
- **pattern:** pattern — A regular expression the value must match. Example: pattern="[0-9]{6}" for a 6-digit PIN.
- **placeholder:** placeholder — Hint text displayed inside the field when empty. This is not a substitute for a <label>.

Example: A full accessible registration form

```
<form action="/register" method="POST">
  <fieldset>
    <legend>Create Your Account</legend>

    <label for="username">Username</label>
    <input type="text" id="username" name="username"
      minlength="3" maxlength="20" required
      pattern="[a-zA-Z0-9_]+"
      aria-describedby="username-hint">
    <small id="username-hint">Letters, numbers, underscores only.</small>

    <label for="email">Email Address</label>
    <input type="email" id="email" name="email" required>

    <label for="password">Password</label>
    <input type="password" id="password" name="password"
      minlength="8" required>

    <label for="dob">Date of Birth</label>
    <input type="date" id="dob" name="dob">

    <label>
      <input type="checkbox" name="terms" required>
      I agree to the Terms and Conditions
```

```
</label>
```

```
<button type="submit">Create Account</button>
```

```
</fieldset>
```

```
</form>
```

Accessibility Basics

Web Accessibility

Web accessibility means designing and building websites so that people with disabilities — visual, auditory, motor, or cognitive — can perceive, understand, navigate, and interact with the web equally. Accessibility is not a nice-to-have feature; in many countries it is a legal requirement under legislation like the ADA (USA), EAA (EU), and RPwD Act (India).

WCAG — Web Content Accessibility Guidelines

The global standard for web accessibility is published by the W3C (World Wide Web Consortium) as WCAG. It organises accessibility requirements under four core principles, remembered with the acronym POUR:

- **Perceivable:** Perceivable — All information must be presentable to users in ways they can perceive. Example: providing text alternatives for images.
- **Operable:** Operable — All interface components and navigation must be operable by all users. Example: everything must be accessible via keyboard alone.
- **Understandable:** Understandable — Content and interfaces must be understandable. Example: form errors must be clearly described, not just highlighted in red.
- **Robust:** Robust — Content must be robust enough to be interpreted reliably by a wide variety of assistive technologies.

ARIA — Accessible Rich Internet Applications

ARIA attributes are a W3C specification that allows developers to add semantic meaning to elements that lack native HTML semantics — most commonly custom interactive widgets built from generic divs and spans:

```
<!-- Announcing page sections to screen readers -->
```

```
<nav aria-label="Primary Navigation">...</nav>
```

```
<!-- Describing a custom dropdown state -->
<button aria-expanded="false" aria-controls="menu">Open Menu</button>
<ul id="menu" hidden>...</ul>

<!-- Linking a hint to its input -->
<input type="email" aria-describedby="email-hint">
<small id="email-hint">We'll never share your email.</small>

<!-- Live region for dynamic updates -->
<div aria-live="polite" id="status">Saving...</div>
```

Alternative Text for Images

All meaningful images must have an alt attribute describing the image's content and purpose. Decorative images that add no informational value should use an empty alt="" so screen readers skip them:

```
<!-- Meaningful image — describe its content and context -->


<!-- Decorative image — empty alt tells screen readers to skip it -->


<!-- Icon button — label comes from aria-label -->
<button aria-label="Close dialog">
  
</button>
```

Keyboard Accessibility

All interactive content — links, buttons, form fields, custom components — must be operable with a keyboard alone. Users who cannot use a mouse (due to motor disabilities, or because they are power users who prefer the keyboard) navigate using Tab, Shift+Tab, Enter, Space, and arrow keys.

- **Use native elements:** Use native HTML interactive elements wherever possible — <button>, <a>, <input>, <select> — they have built-in keyboard behaviour for free.

- **Visible focus styles:** Never remove the focus outline with `outline: none` without providing a custom replacement — it is the only way keyboard users know which element is currently focused.
- **tabindex:** `tabindex="0"` adds a custom element into the natural tab order. `tabindex="-1"` removes it from the tab order while allowing it to be programmatically focused via JavaScript.
- **Focus trapping:** Modal dialogs must trap focus — Tab and Shift+Tab should cycle only within the open modal, not reach behind it to the background page.

```
/* Always provide a visible custom focus indicator */
```

```
button:focus-visible {  
  outline: 3px solid #1A6B7A;  
  outline-offset: 2px;  
}
```

```
/* Never do this — removes the only focus indicator for keyboard users */
```

```
button:focus { outline: none; } /* BAD */
```

Colour Contrast and Visual Design

Text must have sufficient contrast against its background so users with low vision or colour-vision deficiencies can read it. WCAG defines specific ratios:

Content Type	Required Contrast Ratio	Level
Normal text (under 18pt / 14pt bold)	4.5 : 1 minimum	AA
Large text (18pt+ or 14pt+ bold)	3 : 1 minimum	AA
UI components and icons	3 : 1 minimum	AA
Normal text — enhanced	7 : 1 minimum	AAA

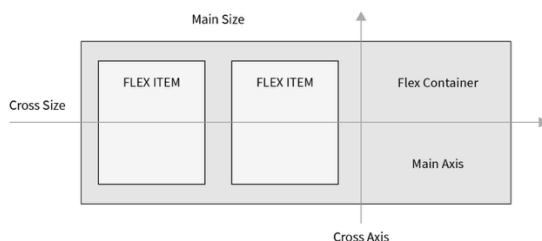
Layout Design — CSS Flexbox

CSS Flexbox

Flexbox (Flexible Box Layout) is a one-dimensional CSS layout model — it lays items out along a single axis at a time, either horizontally (a row) or vertically (a column). It solves classic layout problems — centring content, distributing space, reordering elements — that were notoriously difficult with older float-based techniques.

The Flex Container and Flex Items

Flexbox operates through a two-level relationship: a container element becomes a flex container by setting `display: flex` or `display: inline-flex`. All direct children of that container automatically become flex items. Properties set on the container control the overall layout strategy; properties set on individual items control how each item participates in that layout.



```
.container {  
  display: flex;  
}  
/* All direct children are now flex items */
```

Container Properties

Property	Values	Effect
<code>flex-direction</code>	<code>row</code> (default) <code>column</code> <code>row-reverse</code> <code>column-reverse</code>	Sets the main axis direction
<code>flex-wrap</code>	<code>nowrap</code> (default) <code>wrap</code> <code>wrap-reverse</code>	Whether items wrap to new lines
<code>justify-content</code>	<code>flex-start</code> <code>flex-end</code> <code>center</code> <code>space-between</code> <code>space-around</code> <code>space-evenly</code>	Alignment on main axis
<code>align-items</code>	<code>stretch</code> <code>flex-start</code> <code>flex-end</code> <code>center</code> <code>baseline</code>	Alignment on cross axis (one line)
<code>align-content</code>	Same as <code>justify-content</code> values	Cross-axis alignment for multiple lines
<code>gap</code>	any length (e.g. 16px, 1rem)	Space between items (replaces margin hacks)

Item Properties

Property	Description	Example
flex-grow	How much the item grows to fill remaining space (default: 0)	flex-grow: 1 (fills available space)
flex-shrink	How much the item shrinks when container is too small (default: 1)	flex-shrink: 0 (never shrink)
flex-basis	The item's initial size before growing/shrinking	flex-basis: 200px or auto
flex	Shorthand for grow, shrink, basis	flex: 1 1 auto or flex: 0 0 200px
align-self	Overrides align-items for this specific item	align-self: center
order	Changes the visual order without changing HTML order	order: -1 (move to front)

Common Flexbox Patterns

Example: Perfect centring — both horizontal and vertical

```
.center-me {  
  display: flex;  
  justify-content: center; /* horizontal */  
  align-items: center;    /* vertical */  
  height: 100vh;         /* full viewport height */  
}
```

Example: Navigation bar with logo left, links right

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 0 24px;  
}
```

```
.nav-links {  
  display: flex;  
  gap: 24px;  
  list-style: none;  
}
```

Example: Card grid with equal-width cards

```
.card-row {  
  display: flex;  
  gap: 20px;  
  flex-wrap: wrap; /* allow wrapping on small screens */  
}  
  
.card {  
  flex: 1 1 280px; /* grow, shrink, min-width 280px */  
}
```

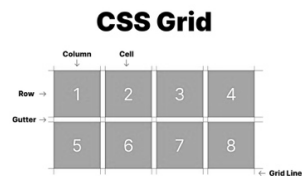
Example: Holy Grail layout — header, 3 columns, footer

```
.page {  
  display: flex;  
  flex-direction: column;  
  min-height: 100vh;  
}  
  
.main-area {  
  display: flex;  
  flex: 1; /* fill remaining space */  
}  
  
.sidebar { flex: 0 0 220px; }  
.content { flex: 1; }
```

Layout Design — CSS Grid

CSS Grid

CSS Grid Layout is a two-dimensional layout system — it handles both rows and columns simultaneously. While Flexbox is ideal for components (a navbar, a row of cards), Grid is ideal for page-level layouts, complex magazine-style designs, and any structure where both axes need explicit control.



Defining the Grid

```
.layout {  
  display: grid;  
  
  /* Define 3 equal columns */  
  grid-template-columns: 1fr 1fr 1fr;  
  
  /* Or using repeat() shorthand */  
  grid-template-columns: repeat(3, 1fr);  
  
  /* Mix fixed and flexible */  
  grid-template-columns: 220px 1fr 1fr;  
  
  /* Define 3 rows */  
  grid-template-rows: auto 1fr auto;  
  
  /* Gap between cells */  
  gap: 20px;  
  /* Or separately: row-gap: 20px; column-gap: 16px; */  
}
```

The fr unit (fraction) is the fundamental unit of CSS Grid. 1fr means one equal fraction of the available remaining space after fixed-size columns have been subtracted. Three columns of 1fr each divide the available width into three equal shares.

The repeat() and minmax() Functions

```
/* repeat(count, track-size) */
grid-template-columns: repeat(4, 1fr);    /* 4 equal columns */
grid-template-columns: repeat(4, 200px);  /* 4 fixed 200px columns */

/* minmax(min, max) — column between 250px and 1fr */
grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
/* Creates as many columns as will fit, each at least 250px wide */
/* This is the responsive grid pattern — no media queries needed */
```

auto-fill vs auto-fit: auto-fill creates empty tracks if items run out; auto-fit collapses empty tracks so remaining items stretch to fill the full width. For most responsive grids, auto-fit gives better results.

Placing Items on the Grid

```
/* Place by grid line numbers */
.hero {
  grid-column: 1 / 3; /* start at line 1, end at line 3 (spans 2 cols) */
  grid-row: 1 / 2;    /* start at line 1, end at line 2 (1 row) */
}

/* Using span keyword */
.featured {
  grid-column: span 2; /* span 2 columns from wherever placed */
  grid-row: span 2;    /* span 2 rows */
}

/* Place by named areas — more readable */
.page {
  display: grid;
  grid-template-areas:
    'header header header'
```

```
'sidebar content content'
'footer footer footer';
grid-template-columns: 220px 1fr 1fr;
grid-template-rows: auto 1fr auto;
}
.page-header { grid-area: header; }
.page-sidebar { grid-area: sidebar; }
.page-content { grid-area: content; }
.page-footer { grid-area: footer; }
```

Explicit vs Implicit Grid

The explicit grid consists of rows and columns deliberately defined using `grid-template-columns` and `grid-template-rows`. The implicit grid consists of additional rows or columns the browser automatically generates when more items are placed than the explicit grid can accommodate.

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  /* No rows defined — they are implicit */
  grid-auto-rows: 200px; /* control implicit row height */
  gap: 16px;
}
/* Items 1-3 fill explicit row 1, items 4-6 auto-create row 2 at 200px, etc. */
```

Flexbox vs Grid — When to Use Each

CSS Flexbox	CSS Grid
One-dimensional (row OR column at a time)	Two-dimensional (rows AND columns simultaneously)
Best for component-level layout	Best for page-level layout
Content drives layout (items size themselves)	Layout drives content (you define the structure)
Navigation bars, button groups, card rows	Full page layouts, magazine grids, dashboards

Items can wrap to new rows automatically	Items placed into named or numbered grid areas
Easier for equal-height columns	Easier for overlapping elements and complex alignment

RESPONSIVE DESIGN AND MODERN STYLING

Responsive Design Using Media Queries

Responsive Web Design

Responsive Web Design (RWD) is the approach of building websites that detect the user's environment (screen width, orientation, resolution) and adapt their layout and appearance accordingly — so a single HTML document renders well on a phone, a tablet, and a desktop monitor without needing separate mobile and desktop versions of the site.

Mobile-First Design Philosophy

Mobile-first is the design and coding strategy of starting with the smallest screen (mobile) as the default, then progressively adding layout complexity for larger screens. This is the opposite of the historical desktop-first approach where developers designed for large screens and then tried to squeeze the layout down.

Mobile-first is the recommended approach for three reasons:

- **Performance:** Performance — Mobile devices often have slower CPUs and slower network connections. Starting mobile-first forces you to prioritise the most essential content and avoid heavy assets that slow mobile loads.
- **Progressive enhancement:** Progressive enhancement — You build a solid foundation that works everywhere, then layer on enhancements for larger screens. This is more robust than trying to gracefully degrade a complex desktop layout.
- **SEO impact:** Search engines — Google uses mobile-first indexing, meaning the mobile version of your site is what Google primarily crawls and ranks.

```
/* Mobile-first: base styles apply to ALL screen sizes */
.card-grid {
  display: flex;
  flex-direction: column; /* stack vertically on mobile */
  gap: 16px;
```

```

}

/* Tablet and up (min-width means 'at least this wide') */
@media (min-width: 640px) {
  .card-grid {
    flex-direction: row;
    flex-wrap: wrap;
  }
}

/* Desktop and up */
@media (min-width: 1024px) {
  .card-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
  }
}

```

Media Query Syntax — Complete Reference

A media query consists of an optional media type and one or more conditions (media features) enclosed in parentheses:

```

/* Basic syntax */
@media [media-type] and ([media-feature]: [value]) { ... }

/* Screen only (excludes print, speech) */
@media screen and (min-width: 768px) { ... }

/* Width range — between 640px and 1280px */
@media (min-width: 640px) and (max-width: 1280px) { ... }

/* Orientation */
@media (orientation: landscape) { ... }
@media (orientation: portrait) { ... }

/* High-resolution displays (retina) */

```

```
@media (-webkit-min-device-pixel-ratio: 2),
  (min-resolution: 192dpi) { ... }

/* Prefers reduced motion — for accessibility */
@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}

/* Dark mode preference */
@media (prefers-color-scheme: dark) {
  body { background: #1a1a1a; color: #ffffff; }
}

/* Modern range syntax (CSS Media Queries Level 4) */
@media (width >= 768px) { ... }
@media (768px <= width <= 1280px) { ... }
```

Common Breakpoints

Breakpoint Name	Width	Typical Target Devices
xs (extra small)	< 480px	Small phones
sm (small)	≥ 480px	Most phones in portrait
md (medium)	≥ 768px	Tablets
lg (large)	≥ 1024px	Laptops and small desktops
xl (extra large)	≥ 1280px	Standard desktops
2xl (2x extra large)	≥ 1536px	Large monitors

Tailwind CSS Breakpoints: Tailwind CSS uses sm (640px), md (768px), lg (1024px), xl (1280px), and 2xl (1536px). These are min-width breakpoints, applied mobile-first. All Tailwind classes without a prefix apply at all screen sizes; prefixed classes (e.g. md:grid-cols-3) apply only at that breakpoint and above.

Responsive Typography

```
/* Base font size for mobile */
body { font-size: 16px; }
h1 { font-size: 1.75rem; } /* 28px */
h2 { font-size: 1.375rem; } /* 22px */

/* Larger type for wider screens */
@media (min-width: 768px) {
  h1 { font-size: 2.25rem; } /* 36px */
  h2 { font-size: 1.75rem; } /* 28px */
}

/* Fluid typography with clamp() — scales between min and max */
/* clamp(minimum, preferred, maximum) */
h1 { font-size: clamp(1.75rem, 5vw, 3rem); }
/* 1.75rem on tiny screens, 3rem on large screens, 5vw in between */
```

Responsive Images

```
/* Always make images responsive by default */
img {
  max-width: 100%; /* never overflow container */
  height: auto; /* maintain aspect ratio */
  display: block; /* remove default inline bottom gap */
}

/* srcset: serve different resolutions */

```

```
/* Modern <picture> for art direction */
<picture>
  <source media="(min-width: 1024px)" srcset="hero-desktop.jpg">
  <source media="(min-width: 640px)" srcset="hero-tablet.jpg">
  
</picture>
```

Introduction to Tailwind CSS

Tailwind CSS

Tailwind CSS is a utility-first CSS framework. Instead of writing semantic CSS class names (.card, .btn-primary) that group related styles, Tailwind provides hundreds of tiny, single-purpose utility classes (flex, text-center, bg-blue-500, p-4, rounded-lg) that you compose directly in your HTML to build any design without ever leaving your markup.

CDN (Content Delivery Network) is a network of servers that delivers web files such as CSS and JavaScript quickly to users.

- In Tailwind CSS, a CDN allows us to use Tailwind CSS directly in an HTML file without installing Node.js, npm, or setting up a Tailwind project.
- We simply include the Tailwind CDN script, such as `<script src="https://cdn.tailwindcss.com"></script>`, and can then use Tailwind utility classes like text-3xl, font-bold, and text-blue-600 directly in HTML.
- It is mainly useful for learning, testing, demonstrations, and small projects.

Adding Tailwind via CDN

For learning, prototyping, and quick projects, Tailwind can be added via a single CDN script tag — no build step, npm, or configuration required:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tailwind Demo</title>
```

```

<!-- Tailwind CSS CDN -->
<script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-50 font-sans">
  <!-- Tailwind utility classes go directly on elements -->
  <h1 class="text-3xl font-bold text-center text-blue-600 mt-8">
    Hello, Tailwind!
  </h1>
</body>
</html>

```

CDN vs Production Build: The CDN version is a JavaScript-based JIT compiler that processes class names at runtime — convenient for learning but not suitable for production due to performance and file size. For production, use the Tailwind CLI or PostCSS plugin to generate a minimal, optimised CSS file containing only the classes you actually use.

Tailwind Core Concept — Utility Classes

Every Tailwind class does exactly one CSS thing. You build components by combining classes:

Category	Classes	CSS Equivalent
Spacing	p-4, pt-2, pb-6, px-8, m-4, mx-auto	padding, margin
Typography	text-sm, text-xl, font-bold, font-semibold, text-center, leading-6	font-size, font-weight, text-align
Colours	text-blue-600, bg-red-100, border-gray-300	color, background-color, border-color
Layout	flex, grid, block, inline, hidden	display
Flexbox	justify-center, items-center, gap-4, flex-wrap	justify-content, align-items, gap, flex-wrap
Grid	grid-cols-3, col-span-2, grid-rows-2	grid-template-columns, grid-column
Sizing	w-full, h-screen, max-w-lg, min-h-0	width, height, max-width

Borders	border, border-2, rounded, rounded-full, rounded-lg	border-width, border-radius
Shadows	shadow-sm, shadow, shadow-lg, shadow-xl	box-shadow
Positioning	relative, absolute, fixed, sticky, top-0, left-4	position, top, left

Tailwind Responsive Prefixes

Every Tailwind utility class can be prefixed with a breakpoint to apply it only at that screen size and above. This is Tailwind's mobile-first responsive system:

```
<!-- Mobile: 1 col | Tablet: 2 cols | Desktop: 4 cols -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">

<!-- Mobile: full width | Desktop: max 1/2 width, centred -->
<div class="w-full lg:w-1/2 lg:mx-auto">

<!-- Mobile: hidden | Tablet+: visible -->
<nav class="hidden md:flex gap-8">

<!-- Mobile: stacked | Desktop: side by side -->
<div class="flex flex-col md:flex-row gap-4">
```

State Variants — Hover, Focus, Active

```
<!-- hover: prefix — styles on mouse hover -->
<button class="bg-blue-600 hover:bg-blue-700 text-white px-6 py-3 rounded-lg
    transition-colors duration-200">
    Click Me
</button>

<!-- focus: and focus-visible: — keyboard focus styles -->
<input class="border border-gray-300 focus:outline-none focus:ring-2
```

```

        focus:ring-blue-500 rounded px-3 py-2">

<!-- active: — pressed state -->
<button class="bg-blue-600 active:bg-blue-800">Press</button>

<!-- group — style children based on parent hover -->
<div class="group">
  <p class="text-gray-600 group-hover:text-blue-600">Hover the parent!</p>
</div>

<!-- dark: — dark mode variant -->
<div class="bg-white dark:bg-gray-900 text-black dark:text-white">

```

Building a Complete Card Component with Tailwind

```

<div class="bg-white rounded-xl shadow-lg overflow-hidden
  hover:shadow-xl transition-shadow duration-300
  max-w-sm">

  <!-- Card Image -->
  

  <!-- Card Body -->
  <div class="p-6">
    <span class="inline-block bg-blue-100 text-blue-700
      text-xs font-semibold px-3 py-1 rounded-full mb-3">
      New Course
    </span>
    <h3 class="text-xl font-bold text-gray-900 mb-2">
      Frontend Design
    </h3>
    <p class="text-gray-600 text-sm leading-relaxed mb-4">
      Learn HTML5, CSS Grid, Flexbox, and Tailwind CSS from scratch.
    </p>
  </div>
</div>

```

```
</p>
<div class="flex items-center justify-between">
  <span class="text-2xl font-bold text-blue-600">Free</span>
  <button class="bg-blue-600 hover:bg-blue-700 text-white
    font-semibold px-4 py-2 rounded-lg
    transition-colors duration-200">
    Enrol Now
  </button>
</div>
</div>
</div>
```

Tailwind Configuration and Custom Values

When using the CDN version, Tailwind can be configured via an inline script to extend or override the default design system:

```
<script src="https://cdn.tailwindcss.com"></script>
<script>
  tailwind.config = {
    theme: {
      extend: {
        colors: {
          brand: '#1A6B7A',
          'brand-light': '#E8F4F6',
        },
        fontFamily: {
          heading: ['Georgia', 'serif'],
          body: ['Calibri', 'sans-serif'],
        },
        spacing: {
          '18': '4.5rem', /* adds p-18, m-18, etc. */
          '22': '5.5rem',
        },
      },
    },
  },
</script>
```

```
</script>

<!-- Now use custom values -->
<h1 class="text-brand font-heading text-3xl">My Brand</h1>
<div class="bg-brand-light p-18">...</div>
```

WEB CONCEPTS — BROWSERS, HTTP/HTTPS, AND DNS

How Browsers Work

Web Browser

A web browser is a software application that retrieves, interprets, and renders web content. It acts as the runtime environment for all frontend code. Understanding what a browser does internally — and the order in which it does things — is foundational to understanding performance, rendering bugs, and why certain CSS or JavaScript behaves the way it does.

Core Browser Components

Component	Responsibility
User Interface (UI)	The visible shell — the address bar, back/forward buttons, bookmarks bar, tabs, and settings menus. Separate from the rendered page content.
Browser Engine	The bridge between the UI and the rendering engine. Orchestrates actions based on user input (e.g. navigating to a URL).
Rendering Engine	Parses HTML and CSS, builds the DOM and CSSOM, and paints pixels to the screen. Chrome/Edge use Blink; Firefox uses Gecko; Safari uses WebKit.
JavaScript Engine	Parses, compiles, and executes JavaScript. Chrome/Edge use V8; Firefox uses SpiderMonkey; Safari uses JavaScriptCore.
Networking Layer	Handles all HTTP/HTTPS requests — DNS resolution, TCP connections, TLS handshake, sending requests, and receiving responses.

Data Storage	Manages cookies, localStorage, sessionStorage, IndexedDB, and the browser cache.
GPU Compositing	Takes the rendered layers and composites them into the final image displayed on screen, using hardware acceleration.

The Critical Rendering Path

The Critical Rendering Path (CRP) is the sequence of steps the browser must complete before the user sees any content on screen. Understanding it explains why certain optimisations (like deferring scripts or inlining critical CSS) have dramatic performance effects:

- **Step 1:** Parse HTML — The browser reads the HTML byte by byte and constructs the DOM (Document Object Model) — a tree of nodes representing every element on the page.
- **Step 2:** Parse CSS — Simultaneously, the browser parses all CSS (from `<link>` stylesheets, `<style>` blocks, and inline styles) and builds the CSSOM (CSS Object Model).
- **Step 3:** Build the Render Tree — The DOM and CSSOM are combined into the Render Tree, which contains only the nodes that are actually visible (elements with `display:none` are excluded).
- **Step 4:** Layout — The browser calculates the exact position and size of every element in the Render Tree. Also called reflow.
- **Step 5:** Paint — The browser fills in pixels — text, colours, images, borders, shadows.
- **Step 6:** Composite — Paint layers are sent to the GPU and combined (composited) into the final image displayed on screen.

Render-Blocking Resources: CSS blocks rendering — the browser cannot build the Render Tree until the CSSOM is complete, so it waits for all CSS to download and parse. By default, JavaScript also blocks parsing — when the browser encounters a `<script>` tag, it stops parsing HTML until the script downloads and executes. This is why scripts are placed at the bottom of `<body>`, or use the `defer` or `async` attributes.

Reflow vs Repaint — Performance Implications

Reflow (Layout)	Repaint
Recalculates the position and size of elements	Only redraws pixels — no geometry change

Triggered by: width, height, margin, padding, font-size changes	Triggered by: color, background, visibility changes
Can cascade — one element affects others around it	Isolated to the affected element(s)
Expensive — avoid triggering in animation loops	Cheaper than reflow, but still has a cost
Animating: left, top, margin-left → triggers reflow	Animating: opacity, color → triggers repaint
Best practice: use transform instead of left/top	Best practice: use opacity instead of visibility

GPU-Accelerated Animation: transform and opacity are the two CSS properties that modern browsers can animate directly on the GPU compositor thread — bypassing both reflow and repaint entirely. This is why CSS animations using transform: translateX() are dramatically smoother than animating the equivalent left property, especially on lower-powered mobile devices.

HTTP and HTTPS

HTTP — HyperText Transfer Protocol

HTTP is the application-layer communication protocol that defines how messages are formatted and exchanged between a client (usually a browser) and a server. Every time you visit a web page, your browser sends one or more HTTP requests and receives HTTP responses. HTTP is stateless — each request is entirely independent; the server retains no memory of previous requests.

The HTTP Request-Response Cycle

Every web interaction follows the same fundamental pattern:

1. The user types a URL or clicks a link in the browser.
2. The browser performs a DNS lookup to resolve the domain name to an IP address.
3. The browser opens a TCP connection to the server (and for HTTPS, performs a TLS handshake).
4. The browser sends an HTTP request containing a method, a path, headers, and optionally a body.
5. The server processes the request and sends back an HTTP response containing a status code, headers, and optionally a body.
6. The browser receives the response and — for an HTML page — begins the Critical Rendering Path.

HTTP Methods

Method	Purpose	Has Body?	Idempotent?
GET	Retrieve a resource. No side effects.	No	Yes
POST	Create a new resource or submit data.	Yes	No
PUT	Replace a resource entirely.	Yes	Yes
PATCH	Partially update a resource.	Yes	No
DELETE	Remove a resource.	No	Yes
HEAD	Identical to GET but returns headers only (no body). Used to check if resource exists.	No	Yes
OPTIONS	Ask the server what methods it supports. Used in CORS preflight.	No	Yes

HTTP Status Codes

Range	Category	Common Codes
1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client Errors	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server Errors	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

HTTPS — HTTP Secure

HTTPS

HTTPS is HTTP with an additional encryption and authentication layer provided by TLS (Transport Layer Security, the successor to SSL). It encrypts all communication between client and server, preventing eavesdropping, tampering, and impersonation attacks. HTTPS is mandatory for any site handling sensitive data, and is now strongly recommended for all websites — browsers mark HTTP sites as 'Not Secure'.

The TLS Handshake — what happens when a browser connects to an HTTPS server:

1. ClientHello — Browser tells server: 'I support TLS 1.3, here are my cipher suites.'
2. ServerHello — Server responds: 'Let's use TLS 1.3 with AES-256. Here is my certificate.'
3. Certificate Verification — Browser checks the certificate against a trusted Certificate Authority (CA). Verifies the domain matches, the certificate is valid, and not expired.
4. Key Exchange — Browser and server use asymmetric cryptography to securely agree on a shared symmetric session key, without ever transmitting the key itself over the network.
5. Secure Channel Established — All subsequent communication is encrypted with the shared session key using fast symmetric encryption.

HTTP	HTTPS
Data sent as plain text	Data encrypted with TLS
Vulnerable to eavesdropping (man-in-the-middle)	Encrypted — unreadable even if intercepted
No server identity verification	Server identity verified via certificate authority
Port 80 (default)	Port 443 (default)
Browser shows 'Not Secure' warning	Browser shows padlock icon
No HSTS support	Can set HSTS to force HTTPS for future visits
Passwords, cards transmitted in plaintext	Sensitive data protected end-to-end

DNS — Domain Name System

DNS — Domain Name System

DNS is the internet's distributed phone directory. It translates human-readable domain names (like `www.google.com`) into machine-readable IP addresses (like `142.250.183.36`) that computers use to route network traffic. Without DNS, you would need to remember numeric IP addresses for every website you visit.

DNS Resolution — Step-by-Step

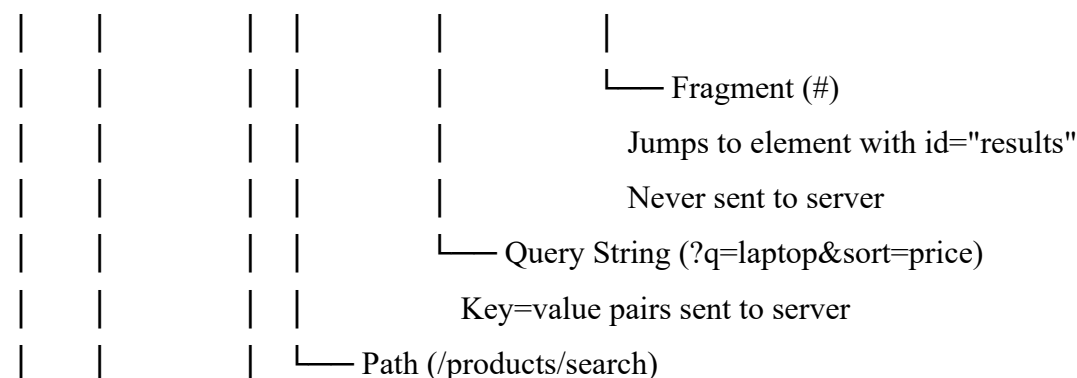
When you type `www.example.com` and press Enter, a chain of lookups occurs:

1. **Browser Cache** — The browser first checks its own DNS cache. If it has recently resolved this domain, it uses the cached result immediately.
2. **Operating System Cache** — If not in the browser cache, the browser asks the OS. The OS checks its own DNS cache (and the `/etc/hosts` file on Linux/Mac).
3. **Recursive Resolver** — If the OS has no answer, it asks the DNS Recursive Resolver (usually provided by your ISP or a public resolver like 8.8.8.8 for Google or 1.1.1.1 for Cloudflare).
4. **Root Nameservers** — The recursive resolver asks one of the 13 root nameserver clusters. The root server responds: 'I don't know, but ask the .com TLD nameserver.'
5. **TLD Nameserver** — The .com TLD nameserver is queried. It responds: 'I don't know the IP, but the authoritative nameserver for example.com is ns1.example.com.'
6. **Authoritative Nameserver** — The authoritative nameserver for example.com is queried. This is the server that holds the actual DNS records. It responds with the IP address: 93.184.216.34.
7. **Response cached and returned** — The recursive resolver caches the result for the duration of the TTL (Time to Live) and returns the IP to the browser.

URL Anatomy

Every URL is structured with specific components that the browser parses before making a request:

https://api.example.com:443/products/search?q=laptop&sort=price#results



| | | Resource path on the server
| | | Port (:443)
| | | Optional; 443 is default for HTTPS, 80 for HTTP
| | | Host (api.example.com)
| | | subdomain.domain.tld
| | | Scheme (https://)
| Protocol: http, https, ftp, mailto, file

References and Textbooks

- 1. Duckett, Jon. HTML & CSS: Design and Build Websites. Wiley, 2011. — Visual, example-driven introduction to HTML structure and CSS styling.
- 2. Robbins, Jennifer Niederst. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics. 5th ed. O'Reilly Media, 2018. — Comprehensive learning resource covering both fundamentals and modern web design practice.
- 3. W3C. Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/> — The global standard for web accessibility.
- 4. MDN Web Docs. Mozilla Foundation. <https://developer.mozilla.org> — The definitive reference for HTML, CSS, JavaScript, and Web APIs.
- 5. Tailwind CSS Documentation. <https://tailwindcss.com/docs> — Official Tailwind CSS reference.

IMPORTANT QUESTIONS

2-MARK QUESTIONS – SHORT ANSWER

- 1) What is semantic HTML? Give any two examples of semantic elements.
- 2) What is the purpose of the `<!DOCTYPE html>` declaration?
- 3) Differentiate between the `<section>` and `<div>` elements.
- 4) What is the purpose of the `<main>` element in an HTML5 webpage?
- 5) What is web accessibility? Mention any two accessibility practices.
- 6) What is the purpose of the `alt` attribute in an HTML image?
- 7) What is the purpose of the `<label>` element in an HTML form?
- 8) What is Flexbox? State whether it is a one-dimensional or two-dimensional layout system.
- 9) What is CSS Grid? Mention one practical use of CSS Grid.

- 10) Differentiate between justify-content and align-items in Flexbox.
- 11) What is responsive web design? Why is it important?
- 12) What is a media query? Write its basic syntax.
- 13) What is Tailwind CSS? Mention any two features of Tailwind CSS.
- 14) What is HTTP? What is the difference between HTTP and HTTPS?
- 15) What is DNS? Why is DNS required when accessing a website?

5/6-MARK QUESTIONS – LONG ANSWER

- 1) Explain the basic structure of an HTML5 document. Discuss the purpose of `<!DOCTYPE html>`, `<html>`, `<head>`, `<meta>`, `<title>` and `<body>` with a suitable example.
- 2) Explain HTML5 semantic elements such as `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>` and `<footer>` with their uses.
- 3) Explain HTML forms and their important attributes. Discuss action, method, enctype and different types of `<input>` elements with suitable examples.
- 4) Explain accessibility in web design. Discuss the importance of labels, alternative text, keyboard accessibility and ARIA attributes.
- 5) Explain CSS Flexbox and its major properties such as flex-direction, justify-content, align-items, flex-wrap and gap with examples.
- 6) Explain CSS Grid layout. Discuss grid-template-columns, grid-template-rows, gap, repeat() and minmax() with suitable CSS examples.
- 7) Differentiate between Flexbox and CSS Grid. Explain their characteristics and mention suitable situations where each should be used.
- 8) Explain responsive web design and the mobile-first approach. Demonstrate how media queries can be used to create a responsive webpage.
- 9) Explain Tailwind CSS and its utility-first approach. Show how Tailwind CSS can be integrated into an HTML page using CDN with a simple example.
- 10) Explain the basic working of a web browser. Describe how a browser processes HTML and CSS and renders a webpage on the screen.

8-MARK QUESTIONS – DETAILED LONG ANSWER

- 1) Explain HTML5 semantic elements and their importance in designing a well-structured webpage. Illustrate with a suitable HTML example.

- 2) Explain the structure of an HTML5 document and describe the purpose of its major elements with a suitable example.
- 3) Explain HTML forms and discuss different input types, form attributes, labels, and HTML5 validation attributes with suitable examples.
- 4) Explain web accessibility and discuss the importance of alternative text, labels, keyboard accessibility, ARIA and colour contrast in web design.
- 5) Explain CSS Flexbox and its major properties. Design a suitable webpage layout using Flexbox with appropriate CSS code.
- 6) Explain CSS Grid layout and discuss grid rows, columns, gaps, fractional units, repeat() and minmax() with suitable examples.
- 7) Differentiate between CSS Flexbox and CSS Grid with respect to layout direction, features and applications. Explain their use with suitable examples.
- 8) Explain responsive web design and the mobile-first approach. Discuss the role of media queries in developing responsive webpages with suitable examples.
- 9) Explain Tailwind CSS and its utility-first approach. Describe the procedure for integrating Tailwind CSS using CDN and illustrate with a suitable example.
- 10) Explain the working of a web browser from entering a URL to displaying the webpage, including HTML parsing, CSS processing, layout, painting and rendering.
- 11) Explain HTTP and HTTPS and compare them with respect to security, encryption, communication and commonly used ports.
- 12) Explain the Domain Name System (DNS) and describe the step-by-step process of DNS resolution when a user enters a website address in a browser.
- 13) Explain the complete process involved when a user accesses an HTTPS webpage, including DNS resolution, HTTP request-response, TLS and browser rendering.
- 14) Explain how Flexbox, CSS Grid and media queries can be combined to design a responsive and maintainable webpage. Illustrate with suitable CSS code.
- 15) Design a responsive student registration webpage using semantic HTML5, accessible forms, CSS layout techniques and Tailwind CSS. Explain the major elements and styling techniques used.